2) RANDOM INITIALIZATION (AND GATE)

```
import numpy as np
#Input data
x1 = np.array([0,0,1,1])
x2 = np.array([0,1,0,1])
t = np.array([0,0,0,1])
m=len(t)
max_iter = 4000
Learn_rate = 0.1
#Initialize weights and bias
W1 = np.random.uniform(-1,1)* 0.2
W2 = np.random.uniform(-1,1)* 0.2
b=0
#sigmoid function
def sigmoid(z):
    return 1/(1+np.exp(-z))
#sigmoid derivative
def sigmoid_derivative(s):
    return s*(1-s)
#Training loop
for _ in range(max_iter):
    dw1 = 0
    dw2 = 0
    db = 0
    for i in range(m):
        z = x1[i] * W1 + x2[i] * W2 + b
        y = sigmoid(z)
        error = t[i] - y
        derivative_y = sigmoid_derivative(y)
        dw1 += error * derivative_y * x1[i]
        dw2 += error * derivative_y * x2[i]
        db += error * derivative_y
    W1 += Learn_rate * (dw1/m)
    W2 += learn_rate * (dw2/m)
    b += learn_rate * (db/m)
print("W1:", W1, "W2:", W2, "b:", b)
final_z = x1 * W1 + x2 * W2 + b
prediction = sigmoid(final_z)
print("prediction:", prediction.round())
```

OUTPUT

```
W1: 2.6104626410594394
W2: 2.6100986924853133
b: -4.0916445576442555
prediction: [0. 0. 0. 1.]
```